

## JOBSHEET PRAKTIKUM

Mata Kuliah: Pemrograman Framework

Pertemuan: Dynamic Routing & Static Generation

Topik: Detail Produk (CSR, SSR, SSG)

---

### A. Capaian Pembelajaran (CPMK)

Mahasiswa mampu:

1. Membuat halaman detail produk menggunakan Dynamic Routing.
  2. Mengambil parameter URL pada Next.js.
  3. Mengimplementasikan detail produk dengan:
    - Client Side Rendering (CSR)
    - Server Side Rendering (SSR)
    - Static Site Generation (SSG)
  4. Menggunakan `getStaticPaths` pada dynamic SSG.
  5. Menganalisis perbedaan performa ketiga metode rendering.
- 

### B. Dasar Teori

#### 1 Dynamic Routing

Dynamic routing pada Next.js menggunakan tanda kurung siku:

```
bash

pages/products/[product].tsx
```

Jika URL:

```
/products/123
```

Maka nilai `product` = 123

---

## 2 Rendering pada Halaman Detail

| Metode Cara Ambil ID            | Cara Fetch Data    |
|---------------------------------|--------------------|
| CSR     useRouter()             | useSWR / fetch     |
| SSR     context.params          | getServerSideProps |
| SSG     params + getStaticPaths | getStaticProps     |

## 3 getStaticPaths

Digunakan untuk:

- Mendaftarkan daftar ID yang akan dibuat secara statis saat build
- Wajib pada dynamic SSG

Contoh:

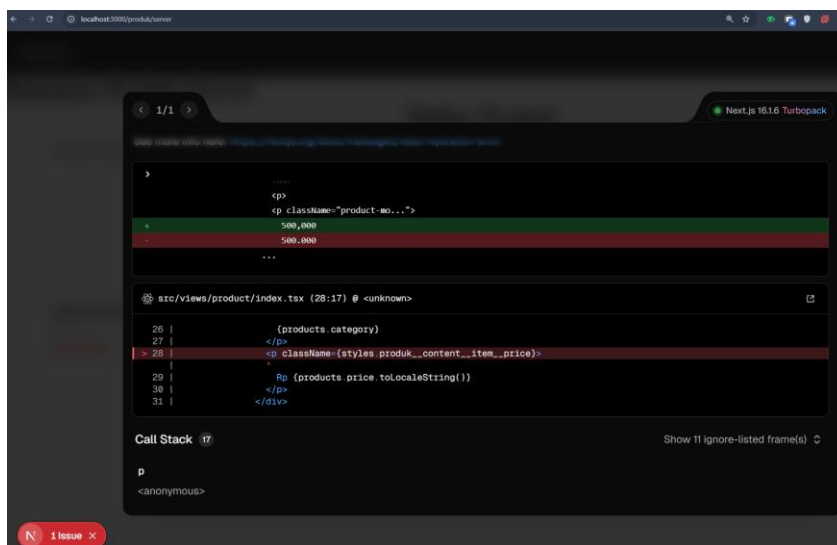
```
export async function getStaticPaths() {
  const res = await fetch('http://localhost:3000/api/products')
  const data = await res.json()

  const paths = data.map((product) => ({
    params: { product: product.id }
  })))

  return { paths, fallback: false }
}
```

Error

Jika saat menjalankan <http://localhost:3000/produk/server> muncul error sbb



Maka buka file index.tsx pada folder views/product/



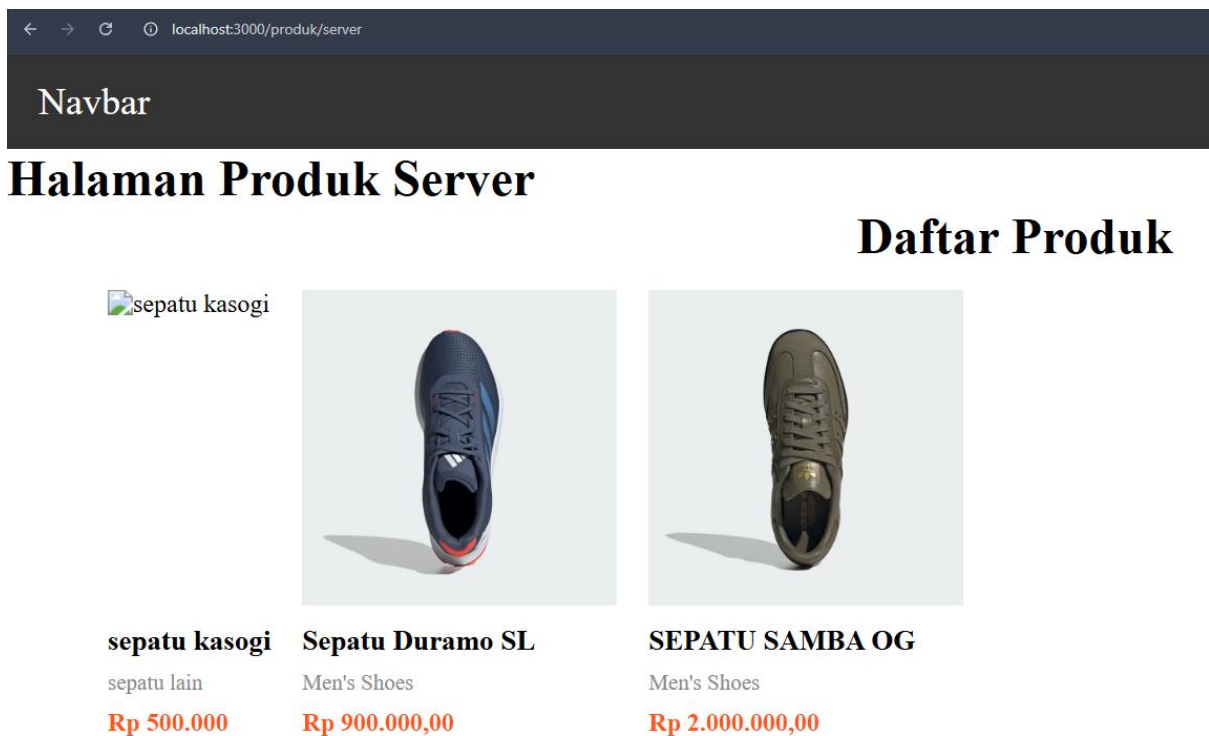
Modifikasi pada code berikut

```
<p className={styles.produk__content__item__price}>  
  Rp {products.price.toLocaleString()}  
</p>
```

Menjadi :

```
<p className={styles.produk__content__item__price}>  
  Rp {products.price.toLocaleString("id-ID")}  
</p>
```

Jalankan <http://localhost:3000/produk/server>



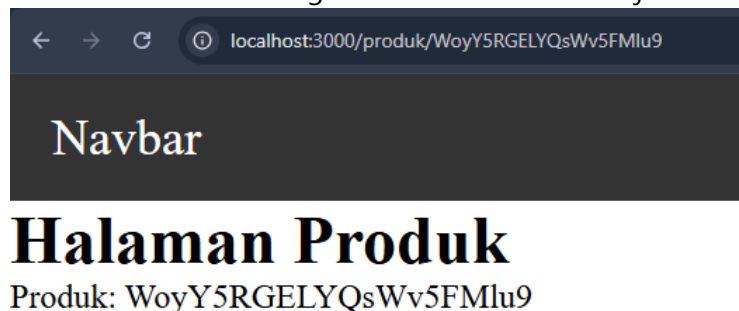
## C. Langkah Praktikum

### ◆ Bagian 1 – Membuat Dynamic Route

- Buka file `pages/products/[product].tsx` dan modifikasi sbb ( line 20 )

```
10.praktikum-rendering_Dynamic_route > my-app > src > views > product > index.tsx > TampilanProduk > products.map() callback
● 1 import styles from "../../pages/produk/product.module.scss";
  2 import Link from "next/link";
  3
  Windsurf: Refactor | Explain
  4 type ProductType = {
  5   id: string;
  6   name: string;
  7   price: number;
  8   image: string;
  9   category: string;
 10 };
 11
  Windsurf: Refactor | Explain | Generate JSDoc | X
 12 const TampilanProduk = ({ products }: { products: ProductType[] }) => {
 13   return (
 14     <div className={styles.produk}>
 15       <h1 className={styles.produk__title}>Daftar Produk</h1>
 16       <div className={styles.produk__content}>
 17         {products.length > 0 ? (
 18           <>
 19             {products.map((products: ProductType) => (
 20               <Link href={` /produk/${products.id}`} key={products.id} className={styles.produk__content_item}>
 21                 <div className={styles.produk__content_item_image}>
 22                   <img src={products.image} alt={products.name} width={200} />
 23                 </div>
 24                 <h4 className={styles.produk__content_item_name}>
 25                   {products.name}
 26                 </h4>
 27                 <p className={styles.produk__content_item_category}>
 28                   {products.category}
 29                 </p>
 30                 <p className={styles.produk__content_item_price}>
 31                   Rp {products.price.toLocaleString("id-ID")}
 32                 </p>
 33               </Link>
 34             )]}
 35           </>
 36         ) : (
 37           <div className={styles.produk__content_skeleton}>
 38             <div className={styles.produk__content_skeleton_image}></div>
 39             <div className={styles.produk__content_skeleton_name}></div>
 40             <div className={styles.produk__content_skeleton_category}></div>
 41             <div className={styles.produk__content_skeleton_price}></div>
 42           </div>
 43         )}
 44       </div>
 45     </div>
 46   );
 47 };
 48 export default TampilanProduk;
```

- Jalankan browser <http://localhost:3000/produk>
  - o Jika kita klik salah satu gambar maka akan menuju halaman lain

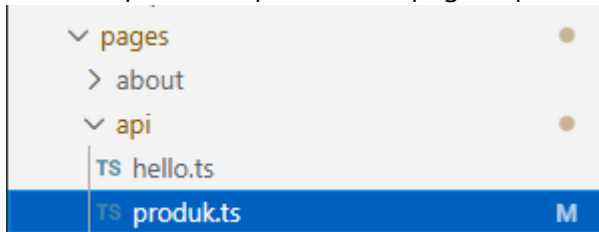


## ♦ Bagian 2 – Implementasi CSR (Client Rendering)

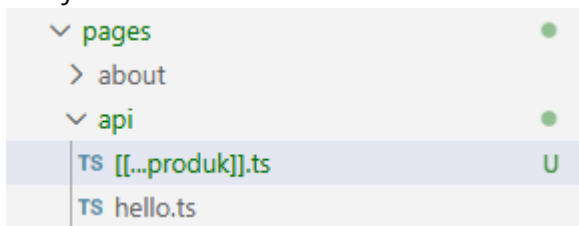
- Modifikasi pada file [produk].tsx pada folder src/pages/produk/

```
10.praktikum-rendering_Dynamic_route > my-app > src > pages > produk > [produk].tsx > ...
1 | import fetcher from "@utils/swr/fetcher";
2 | import { useRouter } from "next/router";
3 | import useSWR from "swr";
4 |
5 |
Windsurf: Refactor | Explain | Generate JSDoc | X
6 | const HalamanProduk = () => {
7 |   // const Router = useRouter();
8 |   // console.log(Router);
9 |   const { query } = useRouter();
10 |   const {data,error,isLoading} = useSWR(`/api/products/${query.produk}`,fetcher);
11 |   return (
12 |     <div>
13 |       <h1>Halaman Produk</h1>
14 |       <p>Produk: {query.produk}</p>
15 |     </div>
16 |   );
17 | };
18 |
19 | export default HalamanProduk;
20 |
21 |
```

- Pada file produk.ts pada folder pages/api di rename menjadi [...product].ts



Menjadi



- Modifikasi file servicefirebase.ts

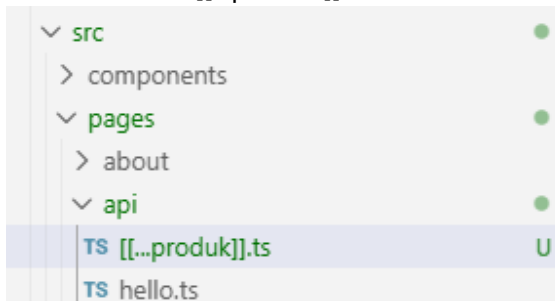


```

10.praktikum-rendering_Dynamic_route > my-app > src > utils > db > TS servicefirebase.ts > retrieveDataById
1  import { getFirestore, collection, getDocs, Firestore, getDoc, doc } from "firebase/firestore";
2  import app from "../firebase";
3
4  const db = getFirestore(app);
5
6  Windsurf: Refactor | Explain | Generate JSDoc | X
7  export async function retrieveProducts(collectionName: string) {
8    const snapshot = await getDocs(collection(db, collectionName));
9    const data = snapshot.docs.map((doc) => ({
10      id: doc.id,
11      ...doc.data(),
12    }));
13    return data;
14  }
15
16  Windsurf: Refactor | Explain | Generate JSDoc | X
17  export async function retrieveDataById(collectionName: string, id: string) {
18    const snapshot = await getDoc(doc(db, collectionName, id));
19    const data = snapshot.data();
20    return data;
21  }

```

- Modifikasi file [...produk].ts

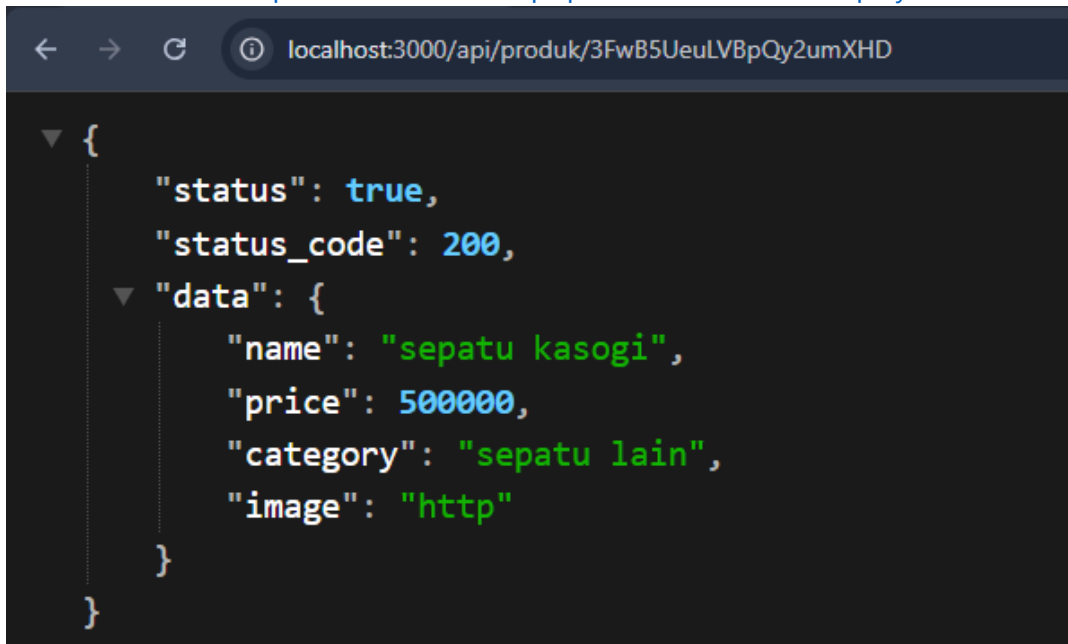


```

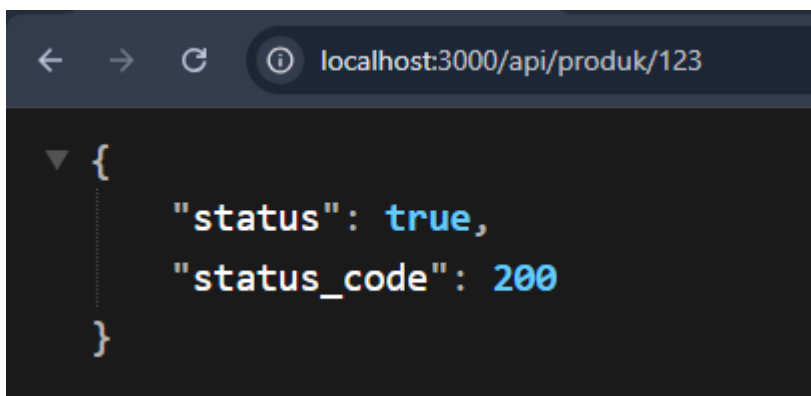
10.praktikum-rendering_Dynamic_route > my-app > src > pages > api > TS [...produk].ts > ...
1  // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2  import type { NextApiRequest, NextApiResponse } from "next";
3  import {
4    retrieveDataById,
5    retrieveProducts,
6  } from "../../utils/db/servicefirebase";
7
8
9  Windsurf: Refactor | Explain
10 type Data = {
11   status: boolean;
12   status_code: number;
13   data: any;
14 };
15
16 Windsurf: Refactor | Explain | Generate JSDoc | X
17 export default async function handler(
18   req: NextApiRequest,
19   res: NextApiResponse<Data>,
20 ) {
21   if (req.query.produk![1]) {
22     const data = await retrieveDataById("products", req.query.produk![1]);
23     res.status(200).json({ status: true, status_code: 200, data });
24     return;
25   } else {
26     const data = await retrieveProducts("products");
27     res.status(200).json({ status: true, status_code: 200, data });
28   }
29 }

```

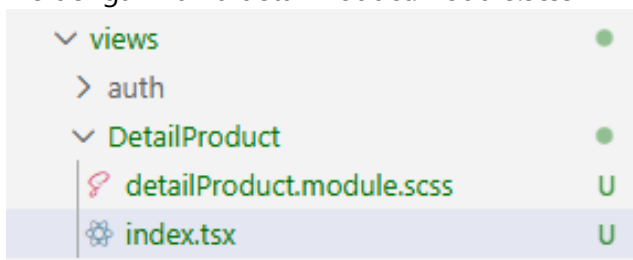
- Jalankan browser <http://localhost:3000/api/produk/3FwB5UeuLVBpQy2umXHD>



- Jalankan alamat url <http://localhost:3000/api/produk/123>



- Buat file dengan nama index.tsx pada folder views/DetailProduct selain itu buat juga file dengan nama detailProduct.module.scss



- Modifikasi detailProduct.module.scss

```
10.praktikum-rendering_Dynamic_route > my-app > src > views > DetailProduct > detailProduct.module.scss > .produkdetail > &_image
29 .produkdetail {
30
31   margin: 40px auto;
32   padding: 24px;
33   display: flex;
34   gap: 40px;
35   background-color: #ffffff;
36   border-radius: 12px;
37   box-shadow: 0 10px 25px rgba(0, 0, 0, 0.08);
38   align-items: flex-start;
39
40   &_image {
41     flex: 1;
42
43     img {
44       width: 100%;
45       border-radius: 12px;
46       object-fit: cover;
47     }
48   }
49
50   &_info {
51     flex: 1.2;
52     display: flex;
53     flex-direction: column;
54   }
55
56   &_name {
57     font-size: 28px;
58     font-weight: 700;
59     margin-bottom: 12px;
60     color: #222;
61   }
62
63   &_category {
64     font-size: 16px;
65     color: #666;
66     margin-bottom: 16px;
67   }
68
69   &_price {
70     font-size: 22px;
71     font-weight: bold;
72     color: #ff5722;
73     margin-bottom: 20px;
74   }
75
76   &_button {
77     width: fit-content;
78     padding: 12px 24px;
79     background-color: #111;
80     color: white;
81     border-radius: 8px;
82     cursor: pointer;
83     transition: 0.3s ease;
84
85     &:hover {
86       background-color: #ff5722;
87     }
88   }
89 }
90
91 /* Responsive */
92 @media (max-width: 768px) {
93   .produkdetail {
94     flex-direction: column;
95
96     &_image {
97       margin-bottom: 20px;
98     }
99   }
100 }
101
```



- Modifikasi index.tsx pada folder DetailProduct

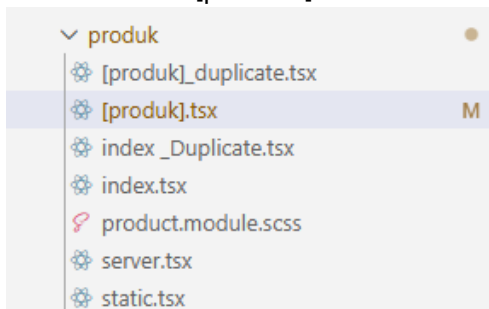
10.praktikum-rendering\_Dynamic\_route > my-app > src > views > DetailProduct > index.tsx > ...

```

1 // import { ProductType } from "@types/Product.type";
2 import { ProductType } from "../../types/Product.type";
3 import styles from "../DetailProduct/detailProduct.module.scss";
4
5 Windsurf: Refactor | Explain | Generate JSDoc | X
6 const DetailProduk = ({ products }: { products: ProductType }) => {
7   return (
8     <div className={styles.produkdetail}>
9       <div className={styles.produkdetail_image}>
10        <img src={products.image} alt={products.name} />
11      </div>
12      <div className={styles.produkdetail_info}>
13        <h1 className={styles.produkdetail_name}>{products.name}</h1>
14        <p className={styles.produkdetail_category}>{products.category}</p>
15        <p className={styles.produkdetail_price}>
16          Rp {products.price.toLocaleString("id-ID")}
17        </p>
18      </div>
19    </div>
20  );
21 };
22
23 export default DetailProduk;

```

- Modifikasi file [produk].tsx



10.praktikum-rendering\_Dynamic\_route > my-app > src > pages > produk > [produk].tsx > HalamanProduk

```

1 import fetcher from "@utils/swr/fetcher";
2 import { useRouter } from "next/router";
3 import useSWR from "swr";
4 import DetailProduk from "../../views/DetailProduct";
5
6 Windsurf: Refactor | Explain | Generate JSDoc | X
7 const HalamanProduk = () => {
8   // const Router = useRouter();
9   // console.log(Router);
10   const { query } = useRouter();
11   const {data,error,isLoading} = useSWR(`/api/products/${query.produk}`,fetcher);
12   return (
13     <div>
14       <DetailProduk products={isLoading ? [] : data.data} />
15     </div>
16   );
17 };
18
19 export default HalamanProduk;
20
21

```

- Modifikasi index.tsx pada folder views/detailProduct line 16



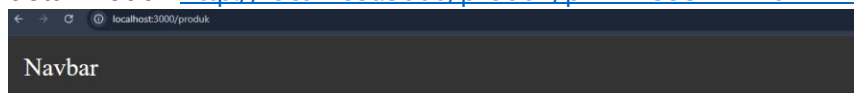
10.praktikum-rendering\_Dynamic\_route > my-app > src > views > DetailProduct > index.tsx > ...

```

1 // import { ProductType } from "@types/Product.type";
2 import { ProductType } from "../../types/Product.type";
3 import styles from "../DetailProduct/detailProduct.module.scss";
4
5 Windsurf: Refactor | Explain | Generate JSDoc | X
6 const DetailProduk = ({ products }: { products: ProductType }) => {
7   return (
8     <>
9     <h1>Detail Produk</h1>
10    <div className={styles.produkdetail}>
11      <div className={styles.produkdetail__image}>
12        <img src={products.image} alt={products.name} />
13      </div>
14      <div className={styles.produkdetail__info}>
15        <h1 className={styles.produkdetail__name}>{products.name}</h1>
16        <p className={styles.produkdetail__category}>{products.category}</p>
17        <p className={styles.produkdetail__price}>
18          Rp {products.price && products.price.toLocaleString("id-ID")}
19        </p>
20      </div>
21    </div>
22    </>
23  );
24 };
25
26 export default DetailProduk;
27

```

- Jalankan browser <http://localhost:3000/produk/> saat produk diklik maka akan muncul detailProduk <http://localhost:3000/produk/pAWIT99SWmVbVrNm49ml>



## Daftar Produk



**Sepatu Duramo SL**

Men's Shoes

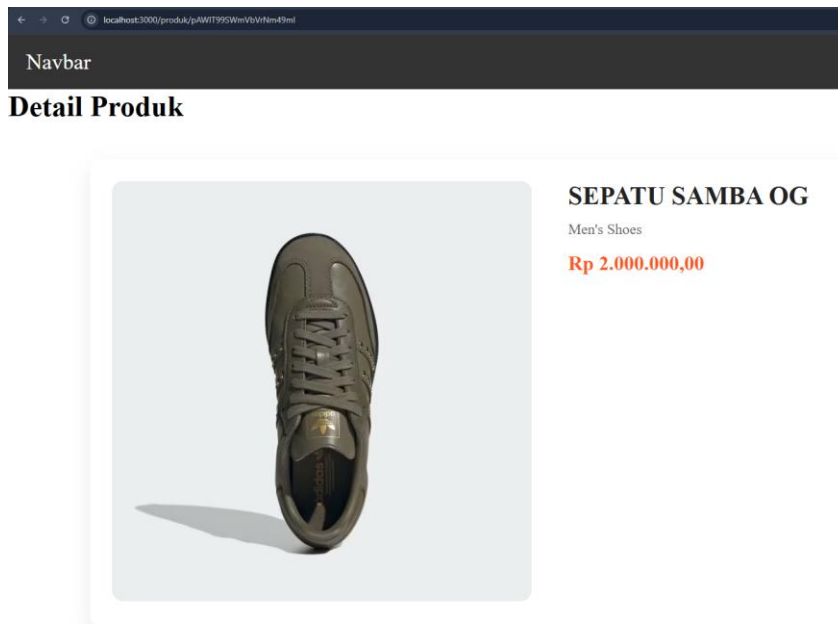
**Rp 900.000,00**



**SEPATU SAMBA OG**

Men's Shoes

**Rp 2.000.000,00**

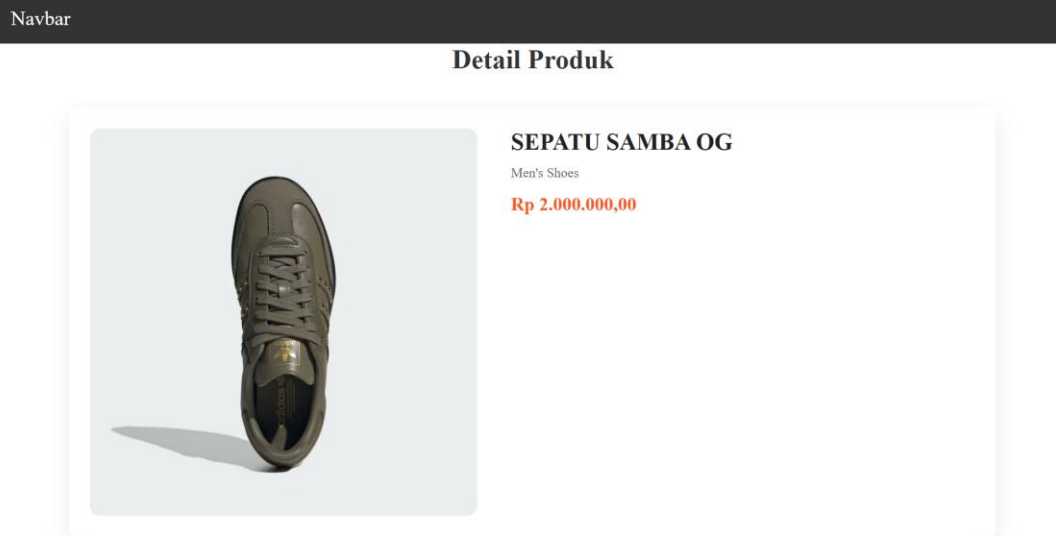


- Agar tulisan detail produk ditengah maka modifikasi file detailProduct.module.scss line 103-108 dan file index.tsx tambahkan code pada line 7,8 dan 22 menjadi

```
10.praktikum-rendering_Dynamic_route > my-app > src > views > DetailProduct > index.tsx > ...
1  // import { ProductType } from "@types/Product.type";
2  import { ProductType } from "../../types/Product.type";
3  import styles from "../../DetailProduct/detailProduct.module.scss";
4
5  Windsurf: Refactor | Explain | Generate JSDoc | X
6  const DetailProduk = ({ products }: { products: ProductType }) => {
7    return (
8      <div className={styles.title}>Detail Produk</div>
9      <div className={styles.produkdetail}>
10        <div className={styles.produkdetail__image}>
11          <img src={products.image} alt={products.name} />
12        </div>
13        <div className={styles.produkdetail__info}>
14          <h1 className={styles.produkdetail__name}>{products.name}</h1>
15          <p className={styles.produkdetail__category}>{products.category}</p>
16          <p className={styles.produkdetail__price}>
17            Rp {products.price && products.price.toLocaleString("id-ID")}
18          </p>
19        </div>
20      </div>
21    </div>
22  </>
23  );
24  };
25
26  export default DetailProduk;
27
```

```
10.praktikum-rendering_Dynamic_route > my-app > src > views > DetailProduct > detailProduct.module.scss > .title
92  @media (max-width: 768px) {
100  }
101
102  .title {
103    text-align: center;
104    font-size: 32px;
105    font-weight: bold;
106    margin-bottom: 40px;
107    color: #333;
108  }
```

- Sehingga hasilnya seperti berikut

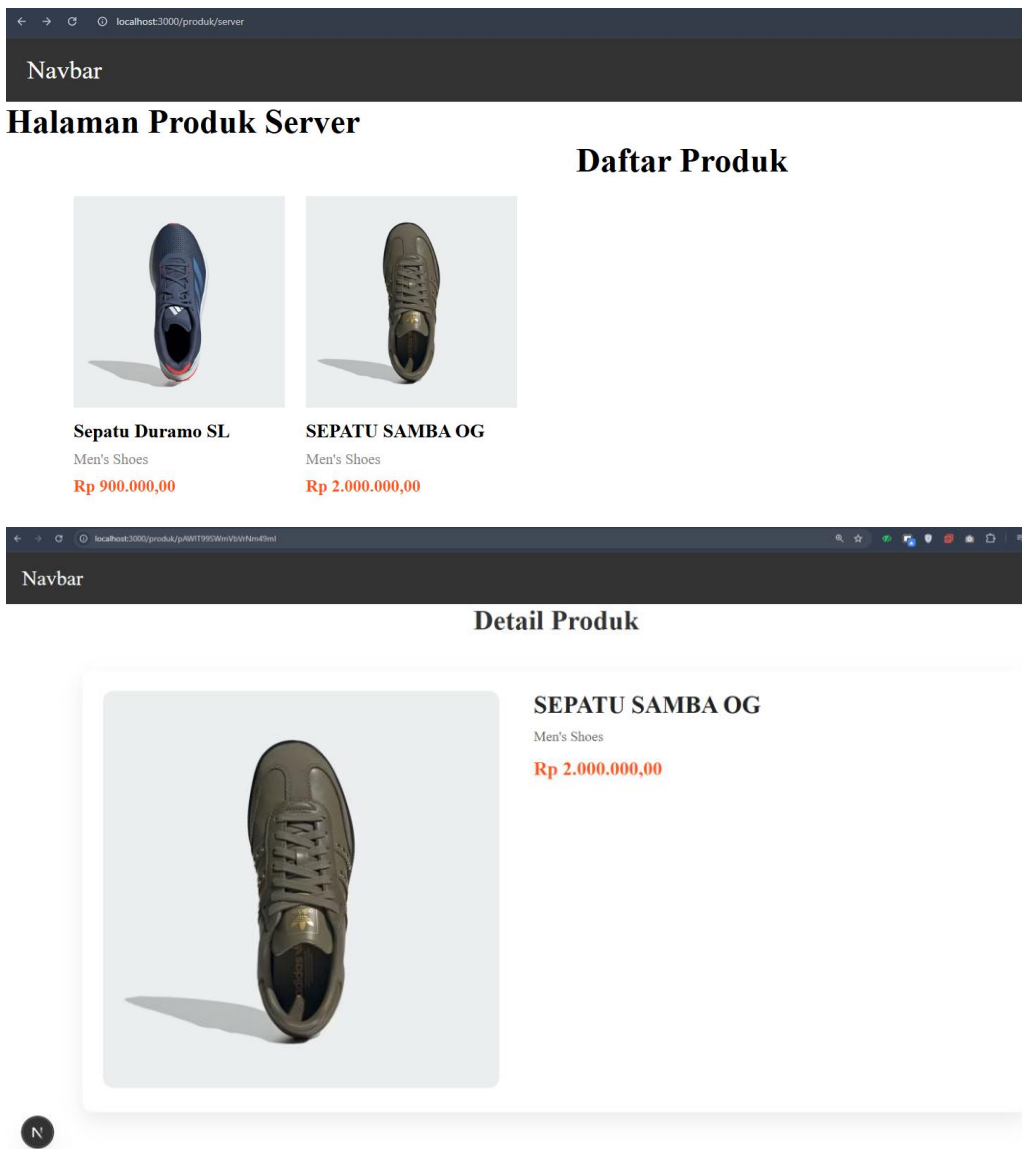


### ◆ Bagian 3 – Implementasi SSR

- Modifikasi [produk].tsx pada folder src/pages/produk dan comment line 9 sampai 20 dikarenakan kita akan menggunakan metode SSR. Tambahkan beberapa kode untuk SSR

```
10.praiktikum-rendering_Dynamic_route > my-app > src > pages > produk > [produk].tsx > getServerSideProps
1 | import fetcher from "@utils/swr/fetcher";
2 | import { useRouter } from "next/router";
3 | import useSWR from "swr";
4 | import DetailProduk from "../../views/DetailProduct";
5 | import { ProductType } from "@types/Product.type";
6 |
7 | Windsurf: Refactor | Explain | Generate JSDoc | X
8 | const HalamanProduk = ({product}: {product: ProductType}) => {
9 |   // // const Router = useRouter();
10 |   // // console.log(Router);
11 |   // const { query } = useRouter();
12 |   // const { data, error, isLoading } = useSWR(
13 |   //   `/api/products/${query.produk}`,
14 |   //   fetcher,
15 |   // );
16 |   // return (
17 |   //   <div>
18 |   //     <DetailProduk products={isLoading ? [] : data.data} />
19 |   //   </div>
20 |   // );
21 |
22 |   return (
23 |     <div>
24 |       <DetailProduk products={product} />
25 |     </div>
26 |   );
27 | };
28 |
29 | export default HalamanProduk;
30 |
31 |
32 | // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
33 | // // digunakan server-side rendering //
34 | Windsurf: Refactor | Explain | Generate JSDoc | X
35 | export async function getServerSideProps({ params }: { params: { produk: string } }) {
36 |   const res = await fetch(`http://localhost:3000/api/produk/${params?.produk}`);
37 |   const response = await res.json();
38 |   // console.log("Data produk yang diambil dari API:", response);
39 |   return {
40 |     props: {
41 |       product: response.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
42 |     },
43 |   };
44 | }
```

- Jalankan browser <http://localhost:3000/produk/server>



✦ Tidak perlu loading state karena data sudah tersedia sebelum render.

---

#### ◆ Bagian 4 – Implementasi Static Site Generation (Dynamic SSG)

- Buka file [produk].tsx dan modifikasi seperti berikut

10.praktikum-rendering\_Dynamic\_route > my-app > src > pages > produk > [produk].tsx > ...

```
1 | import fetcher from "@utils/swr/fetcher";
2 | import { useRouter } from "next/router";
3 | import useSWR from "swr";
4 | import DetailProduk from "../../views/DetailProduct";
5 | import { ProductType } from "@types/Product.type";
6 |
Windsurf: Refactor | Explain | Generate JSDoc | X
7 | const HalamanProduk = ({product}:{product: ProductType}) => {
8 |   {/digunakan client-side rendering/}
9 |   // // const Router = useRouter(); ...
21 |
22 |
23 |   return (
24 |     <div>
25 |       <DetailProduk products={product} />
26 |     </div>
27 |   )
28 | };
29 |
30 | export default HalamanProduk;
31 |
32 |
33 | {/digunakan server-side rendering/}
34 | // //export async function getServerSideProps({ params }: { params: { produk: string } }) {
35 | //   const res = await fetch(`http://localhost:3000/api/produk/${params?.produk}`);
36 | //   const response = await res.json();
37 | //   // console.log("Data produk yang diambil dari API:", response);
38 | //   return {
39 | //     props: {
40 | //       product: response.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
41 | //     },
42 | //   };
43 | // }
44 |
45 |
46 | {/digunakan static-site generation/}
Windsurf: Refactor | Explain | Generate JSDoc | X
47 | export async function getStaticPaths() {
48 |   const res = await fetch('http://localhost:3000/api/products')
49 |   const response = await res.json()
50 |
51 |   const paths = response.data.map((product: ProductType) => ({
52 |     params: { produk: product.id }
53 |   }))
54 |   // console.log("Paths yang dihasilkan untuk produk:", paths); // Debugging: Tampilkan paths yang dihasilkan
55 |   return {
56 |     paths,
57 |     fallback: false
58 |   }
59 | }
60 |
61 |
Windsurf: Refactor | Explain | Generate JSDoc | X
62 | export async function getStaticProps({params}:{params:{produk:string}}) {
63 |   const res = await fetch(`http://localhost:3000/api/produk/${params?.produk}`);
64 |   // const response: ProductType[] = await res.json();
65 |   const response: { data: ProductType[] } = await res.json();
66 |
67 |   // console.log("Data produk yang diambil dari API:", response);
68 |   return {
69 |     props: {
70 |       product: response.data,
71 |     }
72 |   }
73 | }
```

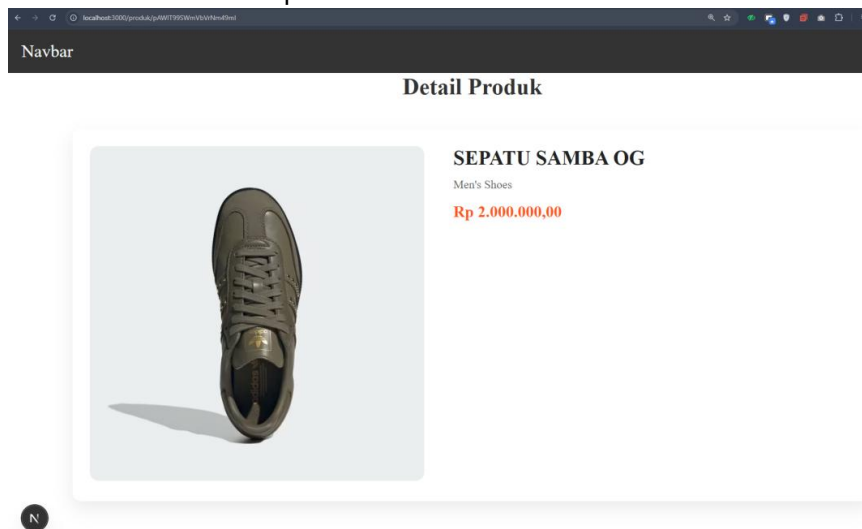
- Buka file index.tsx pada folder src/views/DetailProduct dan modifikasi pada line 11

```
10.praktikum-rendering_Dynamic_route > my-app > src > views > DetailProduct > index.tsx > ...
1 // import { ProductType } from "@types/Product.type";
2 import { ProductType } from "../../types/Product.type";
3 import styles from "../../DetailProduct/detailProduct.module.scss";
4
Windsurf: Refactor | Explain | Generate JSDoc | X
5 const DetailProduk = ({ products }: { products: ProductType }) => {
6   return (
7     <>
8       <h1 className={styles.title}>Detail Produk</h1>
9       <div className={styles.produkdetail}>
10        <div className={styles.produkdetail_image}>
11          <img src={products.image} alt={products.name} />
12        </div>
13        <div className={styles.produkdetail_info}>
14          <h1 className={styles.produkdetail_name}>{products.name}</h1>
15          <p className={styles.produkdetail_category}>{products.category}</p>
16          <p className={styles.produkdetail_price}>
17            Rp {products.price} && products.price.toLocaleString("id-ID")}
18          </p>
19        </div>
20      </div>
21    </div>
22  </>
23 );
24 };
25
26 export default DetailProduk;
27
```

- Jalankan browser <http://localhost:3000/produk>



Saat diklik salah satu produk



## D. Pengujian

---

### Uji 1 – CSR

- Refresh halaman detail
  - Perhatikan loading
  - Periksa Network → XHR → API request terlihat
- 

### Uji 2 – SSR

- Refresh halaman detail
  - Tidak ada loading
  - Periksa Network → tidak terlihat fetch detail
- 

### Uji 3 – SSG ( Lakukan seperti langkah sebelumnya pada Jobsheet 10)

1. Jalankan:
  - npm run build
  - npm run start
2. Tambahkan produk baru di database.
3. Buka halaman detail produk baru:

 Tidak muncul.

4. Build ulang:
  - npm run build
  - npm run start

 Baru muncul.

---

## E. Tugas Praktikum

### Tugas Individu

1. Implementasikan halaman detail dengan:
  - CSR
  - SSR
  - SSG



2. Buat tabel perbandingan:

| Aspek          | CSR | SSR | SSG |
|----------------|-----|-----|-----|
| Loading        |     |     |     |
| Build Required |     |     |     |
| SEO            |     |     |     |
| Perubahan Data |     |     |     |

3. Dokumentasikan:

- Screenshot
- Network tab
- Build result

---

#### F. Pertanyaan Analisis

1. Mengapa `getStaticPaths` wajib pada dynamic SSG?
2. Mengapa CSR membutuhkan loading state?
3. Mengapa SSG tidak menampilkan produk baru tanpa build ulang?
4. Mana metode terbaik untuk halaman detail e-commerce?
5. Apa risiko menggunakan SSG untuk produk yang sering berubah?